

Roteiro de Apresentacao - POO Python

Sistema: Agencia de Emprego Temporario | Python 3.10+ | SQLite + Tkinter | pytest

Fala de abertura sugerida (aprox. 30 s)

"O projeto e um sistema de agencia de emprego temporario desenvolvido em Python com foco nos pilares de POO: heranca, polimorfismo e associacao de classes. Ele possui persistencia em SQLite, interface grafica com Tkinter e cobertura de testes com pytest."

[1,5 pts] 1 - Heranca e Polimorfismo

- Hierarquia: Pessoa (base) -> Profissional (subclasse com dataclass heranca).
- Profissional herda nome e endereco de Pessoa e acrescenta cpf, data_nascimento, profissao.
- `__str__` sobrescrito em Profissional e Empresa - comportamento polimorfismo ao chamar `str()` em listas mistas.
- `Contrato.imprimir_detalhes(contrato, horas)` e o metodo polimorfismo: aceita qualquer contrato e exibe detalhes formatados.

```
# models/pessoa.py
@dataclass
class Pessoa:
    nome: str
    endereco: str

# models/profissional.py
@dataclass
class Profissional(Pessoa): # <- heranca
    cpf: str
    profissao: str
    def __str__(self): return f'{self.nome} ({self.profissao})' # <- polimorfismo
```

Dica: Mostre no terminal: `print(profissional)` -> Joao (Eletricista) - CPF: 123...

[1,0 pt] 2 - Associacao de Classes

- Contrato possui referencias diretas a um Profissional (1:1) e uma Empresa (1:1).
- AgenciaEmprego agrega 3 repositórios (1:N implicito via lista de contratos por profissional).
- Integridade garantida no banco: FOREIGN KEY profissional -> contrato e empresa -> contrato.

```
# models/contrato.py
@dataclass
class Contrato:
    profissional: Profissional # <- associacao 1:1
    empresa: Empresa          # <- associacao 1:1
    valor_hora: float
    ...

# models/agencia.py (AgenciaEmprego)
# agrega RepositorioProfissional, RepositorioEmpresa, RepositorioContrato (1:N)
```

Roteiro de Apresentacao - Agencia de Emprego Temporario (POO)

[1,0 pt] 3 - Tratamento de Excecoes

- Contrato.__post_init__ lanca ValueError para: profissional=None, empresa=None, valor_hora<=0, data_termino < data_inicio.
- RepositorioProfissional.remover captura sqlite3.IntegrityError (FK) e relanca com mensagem clara.
- ConexaoBD._conectar captura sqlite3.Error e relanca como ConnectionError.
- GUI: todos os blocos try/except exibem messagebox.showerror() - erro nunca fica silencioso.

```
def __post_init__(self):
    if self.valor_hora <= 0:
        raise ValueError('valor_hora deve ser maior que zero')
    if date.fromisoformat(self.data_termino) < date.fromisoformat(self.data_inicio):
        raise ValueError('Data de termino nao pode ser anterior a data de inicio')
```

[1,5 pts] 4 - Testes com pytest

- 8 testes em tests/test_agencia.py cobrindo todos os cenarios exigidos:
- calcular_valor_total, valor_hora invalido (0 e negativo), contrato sem profissional, sem empresa.
- __str__ polimorfismo de Profissional e Empresa.
- Persistencia: inserir/recuperar profissional, inserir/recuperar empresa, registrar+listar contrato.
- Integridade referencial: excluir profissional com contrato ativo lanca IntegrityError.
- Fixture conexao_memoria usa SQLite :memory: - banco isolado, sem arquivo em disco.

```
@pytest.fixture
def conexao_memoria():
    con = ConexaoBD(':memory:') # <- banco isolado por teste
    yield con
    con.close()
```

Execucacao:

```
cd agencia_emprego_poo && pytest -q tests/test_agencia.py
```

Dica: Execute os testes ao vivo durante a apresentacao para mostrar que tudo passa.

[2,5 pts] 5 - Persistencia - Banco de Dados

- ConexaoBD usa padrao Singleton: uma unica conexao por caminho de arquivo.
- Banco criado automaticamente no primeiro run: CREATE TABLE IF NOT EXISTS (3 tabelas).
- PRAGMA foreign_keys = ON garante integridade referencial no SQLite.
- Padrao DAO aplicado: RepositorioProfissional, RepositorioEmpresa, RepositorioContrato.
- CRUD completo em profissional e empresa (inserir, buscar, listar, atualizar, remover).
- Contratos sao imutaveis - remover() lanca PermissionError (regra de negocio).
- JOIN no listar_todos de contrato reconstrói objetos Python completos (Profissional + Empresa).

```
# Tabelas criadas automaticamente:
# profissional(cpf PK, nome, endereco, data_nascimento, profissao)
# empresa(cnpj PK, razao_social, endereco)
# contrato(numero AUTOINCREMENT PK,
#         cpf_profissional FK -> profissional,
#         cnpj_empresa FK -> empresa, ...)
```

Roteiro de Apresentacao - Agencia de Emprego Temporario (POO)

[2,0 pts] 6 - Interface Grafica - GUI

- AppGUI organiza 4 abas via ttk.Notebook: Profissionais, Empresas, Contratos, Historico.
- Cada tela e um ttk.Frame independente - separacao de responsabilidades.
- Tela Profissionais: Treeview + formulario modal (Adicionar/Editar/Remover) com validacao de campos vazios.
- Tela Empresas: mesma estrutura (CNPJ, razao social, endereco).
- Tela Contratos: Comboboxes carregados do BD para selecionar profissional e empresa + campos de data e valor.
- Tela Historico: listagem somente-leitura de todos os contratos.
- Todos os erros (campos vazios, BD, FK) sao exibidos via messagebox.showerror() na GUI.
- Ao trocar de aba, todas as listas se atualizam automaticamente (evento NotebookTabChanged).

Dica: Execute python main.py e navegue pelas abas ao vivo: cadastre um profissional, uma empresa e registre um contrato.

[0,5 pt] 7 - Organizacao e README

- Estrutura de pastas clara: models/ (negocio), database/ (DAO), gui/ (apresentacao), tests/.
- requirements.txt minimalista: apenas pytest>=7.0 (SQLite e Tkinter sao nativos do Python).
- README.md com justificativas tecnicas (SQLite, Tkinter), estrutura de pastas, instrucoes de execucao e testes.
- Banco agencia.db criado automaticamente - sem configuracao previa necessaria.

Ordem de Demo ao Vivo (aprox. 5 a 7 min)

1. Abra o terminal na pasta agencia_emprego_poo/
2. Execute: `pytest -q tests/test_agencia.py` -> mostre 8 testes passando
3. Execute: `python main.py` -> abre a janela GUI
4. Aba Profissionais: adicione 1 profissional (ex.: Joao / Eletricista / CPF 12345678900)
5. Aba Empresas: adicione 1 empresa (ex.: Tech Ltda / CNPJ 12345678000190)
6. Aba Contratos: registre um contrato vinculando os dois
7. Aba Historico: mostre o contrato listado
8. Tente remover o profissional -> exibe erro de integridade na GUI
9. Mostre no codigo: Profissional herda Pessoa, Contrato.__post_init__ com ValueError

Pontos-chave para mencionar ao professor

- Heranca real via @dataclass: Profissional(Pessoa) - nao e composicao, e heranca.
- Polimorfismo: `str(objeto)` se comporta diferente em Profissional vs Empresa.
- DAO isola o banco da logica de negocio - AgenciaEmprego nunca toca SQL diretamente.
- FOREIGN KEY ON DELETE RESTRICT impede remocao de dados com vinculo ativo.
- Fixture :memory: garante testes independentes e rapidos sem efeito colateral em disco.
- Singleton em ConexaoBD evita multiplas conexoes ao mesmo arquivo.